

Generalized Matrix and Vector Interface

1. Introduction

A matrix may be stored in different ways depending on its structure. A dense representation stores every element, while structured representations can reduce storage or restrict processing to useful regions. This assignment defines a common abstract Matrix interface so that numerical algorithms can operate on different matrix backends without changing the algorithms.

Aim

To design a generalized Matrix and Vector interface that hides the internal storage representation and supports Dense, CSR, Banded, and Block matrices.

Objectives

- Design a common abstract Matrix interface.
- Support shape, element access, matrix-vector multiplication, and transpose.
- Implement Dense, CSR, Banded, and Block matrix representations.
- Apply backend-independent numerical algorithms.
- Implement Power Iteration and the Jacobi Iterative Method.
- Verify correctness across all matrix backends.
- Measure and compare execution time.

Test problem

All four backends represent the same 4×4 matrix:

4	1	0	0
1	4	1	0
0	1	4	1
0	0	1	4

Input vector: $x = [1, 2, 3, 4]$. Right-hand-side vector: $b = [5, 6, 6, 5]$.

2. GENERALIZED INTERFACE AND IMPLEMENTATIONS

The abstract Matrix class defines `shape()`, `get(i,j)`, `matVec(x)`, and `transpose()`. The `@` operator is overloaded so that `A @ x` performs matrix-vector multiplication. This abstraction separates the algorithms from the storage representation.

Dense Matrix

The DenseMatrix stores all matrix elements in a two-dimensional list. Matrix-vector multiplication visits every row and column. For an $n \times n$ matrix, storage and multiplication are $O(n^2)$.

CSR Matrix

CSR (Compressed Sparse Row) stores only non-zero values using three arrays: Values, Column Index, and Row Pointer. For the test matrix, the stored values are [4, 1, 1, 4, 1, 1, 4, 1, 1, 4], with column indices [0, 1, 0, 1, 2, 1, 2, 3, 2, 3] and row pointer [0, 2, 5, 8, 10].

Banded Matrix

The test matrix has lower bandwidth 1 and upper bandwidth 1. Only positions within these bands are processed during matrix-vector multiplication.

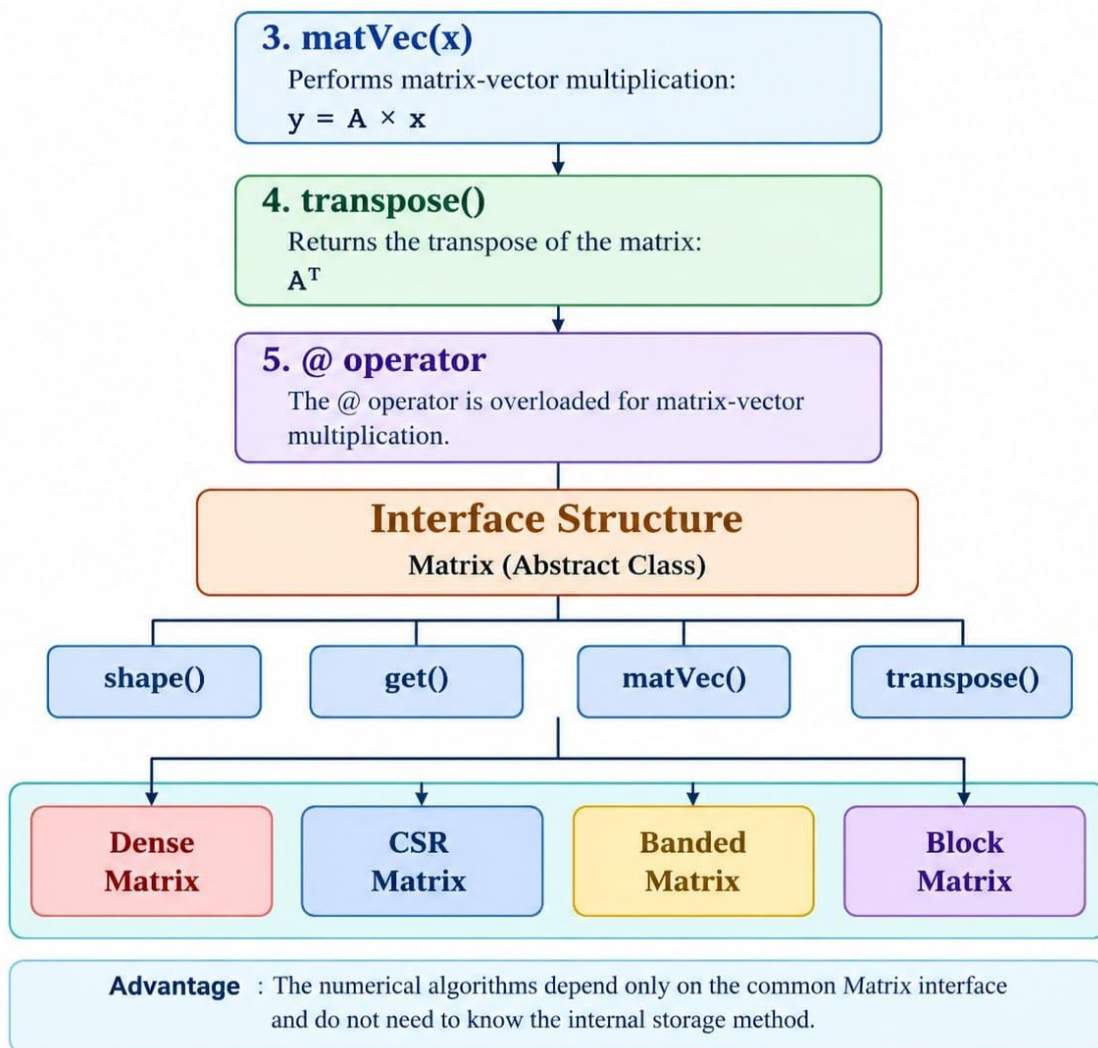
Block Matrix

The BlockMatrix divides the matrix into blocks. The submitted program uses block size 2 and processes the matrix block by block.

Design benefit

Power Iteration, Jacobi, and the correctness tests use the common interface. Therefore, the same algorithms can be applied to all four backends without rewriting the numerical logic.

Interface Structure – Visual Overview



3. NUMERICAL ALGORITHMS

Power Iteration

Power Iteration approximates the dominant eigenvector. The program starts with a vector of ones, normalizes it, repeatedly computes $A \times x$, and normalizes the result. The final vector is used for cross-backend comparison.

Algorithm

- Initialize x .
- Normalize x .
- Compute $x = A \times x$.
- Normalize x .
- Repeat for the requested number of iterations.
- Return x .

Jacobi Iterative Method

The Jacobi method solves $Ax = b$ iteratively. For each row, the non-diagonal contribution is computed and the new variable is obtained from the diagonal element:

$$X(i)_{\text{new}} = (b(i) - \sum_{j \neq i} A(i, j) x(j)) / A(i, i)$$

The implementation starts from a zero vector and repeats the update for a fixed number of iterations. A zero diagonal element raises an error.

Correctness strategy

The Dense Matrix acts as the reference implementation. Each other backend is compared with it for shape, element access, matrix-vector multiplication, transpose, Power Iteration, and Jacobi results. Numerical vector comparisons use a tolerance of $10e-6$ for the iterative algorithm checks.

4. TESTING AND MEASURED RESULTS

The submitted source was executed successfully in the verification environment. The program performs seven checks: shape, get/element access, matrix-vector multiplication, transpose, Power Iteration, Jacobi Solver, and performance.

Correctness results

Test	Dense	CSR	Banded	Block
Shape	PASS	PASS	PASS	PASS
Get	PASS	PASS	PASS	PASS
Matrix-vector	PASS	PASS	PASS	PASS
Transpose	PASS	PASS	PASS	PASS
Power Iteration	PASS	PASS	PASS	PASS
Jacobi Solver	PASS	PASS	PASS	PASS

Matrix-vector reference result

For $x = [1, 2, 3, 4]$, the reference result is $[6.0, 12.0, 18.0, 19.0]$. All four backends produced the same result.

Power Iteration result

The verified result was approximately $[0.371748, 0.601501, 0.601501, 0.371748]$ for all four backends.

Jacobi result

The verified result was $[1.0, 1.0, 1.0, 1.0]$ for all four backends.

These results show numerical agreement across the different storage representations for the supplied test matrix.

5. PERFORMANCE ANALYSIS, TRADE-OFFS AND CONCLUSION

Measured matrix-vector multiplication time

Backend	5000 repetitions (seconds)
Dense	0.008276
CSR	0.006688
Banded	0.006974
Block	0.012425

The timings above are from the verification run and are dependent on the computer, Python version, and runtime conditions. They should therefore be treated as measured observations for this run rather than universal performance values.

Trade-offs

Dense storage is straightforward and performs predictable full-matrix processing, but it stores zeros. CSR reduces storage and work for sparse matrices by storing only non-zero elements. Banded storage is useful when non-zero values are concentrated near the diagonal. Block processing groups operations and can be useful for cache-oriented computation, although the submitted implementation may introduce loop overhead for a small matrix.

Abstraction also has a cost. Method calls, object-oriented dispatch, generic access through `get()`, and unnecessary object creation can add runtime overhead. The assignment therefore demonstrates a balance between reusable design and backend-specific optimization.

Conclusion

A generalized Matrix and Vector interface was implemented with Dense, CSR, Banded, and Block backends. The same numerical algorithms were applied to every backend, and the correctness tests produced PASS results across all tested operations. The performance experiment also demonstrates that storage representation affects execution characteristics. The design is applicable to numerical computing, scientific computing, machine learning, graph algorithms, eigenvalue problems, and sparse linear systems.